AI 서비스 Mini Project | 선택 과제 2 | 최종 보고서

G077 문성도 | 신규 프로젝트

변압기 수주 기술사양 검토 AI

수주 확정 후 기술 인계회의 전에,
변경된 요구사항과 내부 인계표의 차이를 확인합니다.

주제와 주요 task

고객 사양서, 기술 변경 합의서, 내부 수주 인계표를 함께 검토하는 로컬 웹 프로토타입입니다. AI가 근거와 값을 제안하고 코드가 문서 간 차이를 비교합니다. 담당자는 연결된 원문을 확인한 뒤 판단합니다.

구분	구현 내용
사용 시점	수주 확정 후, 상세 설계·구매·제작으로 기술 정보를 넘기기 전입니다.
해결할 문제	여러 문서에 흩어진 요구사항과 승인 변경이 인계표에 누락되거나 다르게 반영되는 상황입니다.
제공 결과	항목별 일치·불일치·누락·확인 필요, 양쪽 원문 근거, 저장·복원, 회의용 보고서입니다.
판단 경계	문서 정합성을 보조합니다. 설계 적합성·제품 안전·실제 승인 및 발주를 결정하지 않습니다.

검토하는 기술사양 10개

정격용량 · 고압측 전압 · 저압측 전압 · 주파수 · 상수 · 결선 그룹 · 냉각 방식 · 팬 보조전원 전압 · 탭절환기 방식 · 옥내형/옥외형

OpenAI 실행 확인 | 2026-10-21
https://github.com/seogdo/transformer-spec-review

01 구현 구조

Agent Graph로 설명하는 실제 Python 제어 흐름

PDF 3종을 읽은 뒤 고객사 표기를 먼저 확인합니다. 고객사가 다르거나 불명확하면 비교를 보류합니다. 같은 고객사로 확인되면 근거 검색과 추출을 진행합니다.

실행 순서	다음 단계와 보류 분기
1 문서 읽기	코드가 텍스트 PDF와 지정 메타데이터를 읽고 문서·페이지·원문을 보존합니다.
2 고객사 확인	일치하면 추출로 이동합니다. 누락·상충·혼입이면 AI 실행과 기술 비교를 보류합니다.
3 계획 → 검색	AI 1회차가 검색 계획을 제안하고 코드가 키워드 검색을 실행합니다. 근거가 없으면 2회차를 생략합니다.
4 제안 → 검증	AI 2회차가 값·단위·인용을 제안합니다. 코드 검증 실패·모호함은 확인 필요로 남깁니다.
5 변경 적용	담당자가 적용할 변경을 확인합니다. 코드는 승인 표기·프로젝트·명시적 대체 관계를 검사합니다.
6 비교 → 저장	코드가 단위를 정규화해 10항목을 비교합니다. 담당자가 원문과 결과를 확인하고 기록을 보관합니다.

설명용 흐름과 구현 기술의 구분

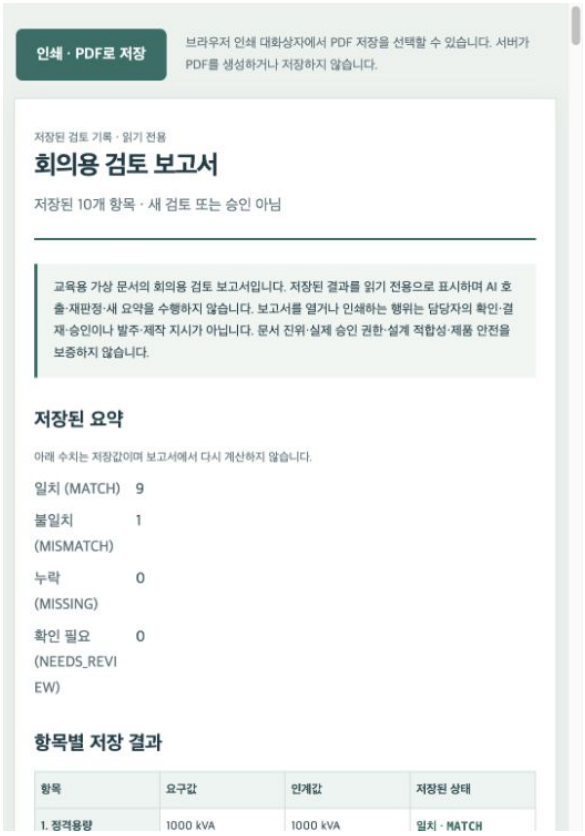
단일 워크플로의 순차 실행·분기를 직접 구현했습니다. LangGraph 실행 엔진이나 Multi-Agent 구조가 아닙니다. 정형 모드는 AI 없이 코드 파서에서 비교로 이어지는 별도 경로입니다.

extract_upload:574 / 검색계획:624 / 검색 실행:632 / 값 제안:643 / 검증:683행입니다. 추출 한 번의 AI 요청은 최대 2회·전체 90초입니다. OpenAI 외부 전송·과금 동의를 확인하고 실행하며, 자동 재시도·제공자 전환은 없습니다. 별도 선택 설명은 추가 호출입니다.

02 대표 시연

440 V 요구와 380 V 인계를 원문으로 확인합니다

V01 가상 변경 합의서에는 팬 보조전원이 440 V로 바뀌었지만, 내부 인계표에는 380 V가 남아 있습니다. 실제 OpenAI API 시험에서 10개 항목 중 9개 일치와 1개 불일치가 표시됐습니다.



OpenAI / gpt-4o-mini / 2026-10-21의 저장된 실제 실행 결과 요약입니다. 제안 21개 중 21개가 코드의 원문 검증을 통과했습니다. 회사 문서나 운영 화면은 아닙니다.

담당자가 확인할 수 있는 정보

요구값과 인계값, 적용된 변경, 문서명·페이지·인용문을 함께 제공합니다. 원본 PDF를 열어 추출 내용과 대조할 수 있으며, 보류된 후보 근거는 확정값과 구분합니다.

오류를 다루는 방식

파일 형식 오류와 AI 연결 실패를 안내하고, 확인되지 않은 값을 임의로 채우지 않습니다. 접수 후 응답 오류가 발생해도 같은 요청으로 기존 작업을 다시 확인하도록 구성했습니다.

03 검증 결과와 한계

동작 시험과 모델 정확도를 구분했습니다

시험	확인 결과
Python 자동 검사	444개 통과. 처리 규칙·오류 처리·보고서의 저장 결과 보존 등을 검사했습니다.
화면 모의 검사	근거 12개, 복구 45개, 보고서 14개 통과. 실제 브라우저 시험과는 별도입니다.
OpenAI V01	21개 제안 중 21개가 원문 검증을 통과했습니다. 9개 일치·1개 불일치입니다.
OpenAI V02	21개 제안 중 20개가 원문 검증을 통과했습니다. 9개 일치·1개 확인 필요이며, 주파수 미기재를 보류했습니다.
OpenAI V03	22개 제안 중 22개가 원문 검증을 통과했습니다. 8개 일치·2개 불일치이며, 탭절환기 방식과 설치 구분의 차이를 확인했습니다.
기존 회의 보고서	기존 Ollama V03 기록을 Chrome에서 9쪽 PDF로 저장했습니다. 새 OpenAI 인쇄 시험 결과가 아닙니다.

결과 해석

자동·화면 모의 검사와 별도로 OpenAI gpt-4o-mini을 실제 호출했습니다. V01~V03은 각 2회, 총 6회 요청이 성공했습니다. 모든 시연 문서는 직접 구성한 가상 개발 자료입니다. 코드 검증 통과 수와 모델의 독립 정확도는 다르며, 이 결과를 실제 회사 문서의 정확도나 시간 절감률로 해석하지 않습니다. 기존 Ollama 실행·인쇄 기록은 새 OpenAI 결과와 구분합니다.

새 실제 실행 요약: reports/openai-validation-20261021.json

현재 제한과 배운 점

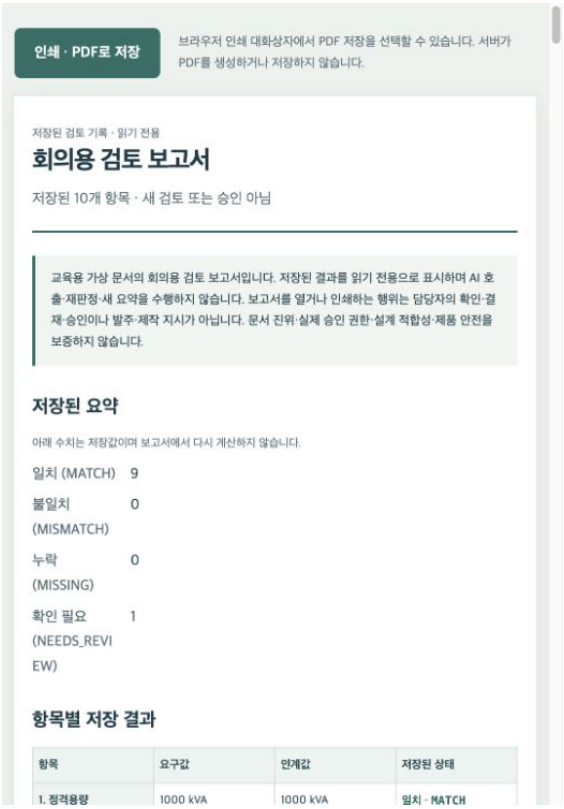
지정 메타데이터와 가상 자료 선언이 있는 텍스트 PDF 3종을 지원합니다. 파일당 5 MiB·20쪽 제한이 있으며 스캔·OCR·도면·다제품 문서는 지원하지 않습니다. 고객사 별칭과 승인 상태는 데모 입력을 확인하는 수준입니다.

이번 구현에서 얻은 교훈은 값을 많이 채우는 것보다, 근거와 불확실성을 함께 보여주는 설계가 중요하다는 점입니다. 테스트 통과와 실제 모델 품질, 원문 연결과 설계 승인도 서로 구분해야 했습니다. 후속 개선은 실제 문서의 독립 평가, 승인 이력·접근 권한 연결, 검색 방식과 호출 수의 동일 입력 비교입니다.

04 추가 실행 화면과 결과 보관

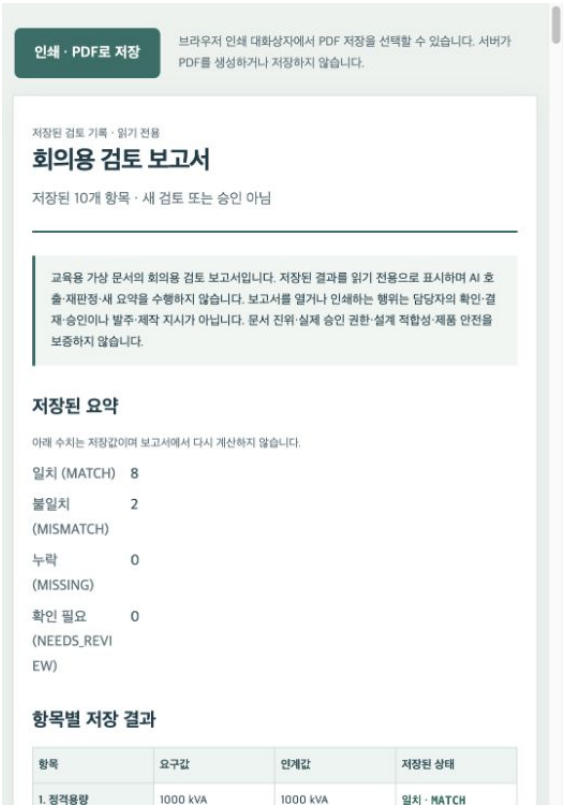
미기재와 사양 차이를 서로 구분했습니다

V02 미기재 보류



21개 제안 중 20개가 원문 검증을 통과했습니다. 9개 일치·1개 확인 필요이며, 주파수 미기재를 보류했습니다.

V03 사양 불일치



22개 제안 중 22개가 원문 검증을 통과했습니다. 8개 일치·2개 불일치이며, 탭절환기 방식과 설치 구분의 차이를 확인했습니다.

두 화면은 OpenAI gpt-4o-mini의 새 실행 결과를 조회한 캡처입니다. 결과와 원문을 확인한 뒤 회의용 보고서를 열거나 JSON을 내려받습니다. 조회·보고서 인쇄는 AI를 다시 호출하지 않습니다.

시연과 제출

앱 실행 → V01 선택 → AI 추출 → 변경 기준 확인 → 10개 항목 비교 → 불일치의 양쪽 원문 확인 → 사람 검토 확인 → 회의용 보고서 열기 입니다.

https://github.com/seogdo/transformer-spec-review
README의 실행 방법과 docs/presentation-demo.md의 시연 순서를 확인합니다. 회사 문서·강의 원본·API 키는 공개 자료에서 제외했습니다.

제출 시트의 E열에는 공개 저장소, F열에는 최종 보고서 공개 주소를 입력해야 합니다. 파일 작성·공개와 시트 제출 완료는 별도이며, 보고서 작성 시점에는 시트 입력을 수행하지 않았습니다.

05 기술 적용과 의존성 주입

사용한 기술과 사용하지 않은 기술을 구분합니다

구분	실제 적용 내용
RAG · AI 연결	OpenAI gpt-4o-mini의 계획·값 제안과 키워드·별칭 근거 검색입니다. Ollama 선택 실행도 지원하며, 벡터 검색은 아닙니다.
DI · 모듈화	request_fn 함수 주입을 사용합니다. PDF 읽기·추출·고객사 확인·단위 처리·비교·웹·보고서를 분리했습니다.
검사 · 기록	Python 444개, 화면 모의 71개 통과 기록입니다. 단계별 trace와 실행 제공자·모델을 기록합니다.
웹 · 저장	Python·FastAPI, HTML·CSS·JavaScript, 로컬 JSON입니다. 공개 저장소는 소스 공유이며 운영 서비스 배포가 아닙니다.
미구현	Multi-Agent, MCP, LangGraph, LangSmith, 임베딩·벡터 DB, 네이티브 도구 호출은 사용하지 않았습니다.

DI: 실제 요청과 시험 응답을 분리했습니다

extract_upload의 600-602행입니다. 주입 함수가 없으면 서버 시작 때 고른 OpenAI 또는 Ollama 요청을 사용합니다. 시험에서는 계획·값 제안을 고정 응답으로 주입합니다.

```
request = request_fn if request_fn is not None else (
    (lambda payload, timeout: openai_request(config, payload, timeout)) if config.provider == "openai" else _local_request)
deadline = time.monotonic() + TOTAL_TIMEOUT
```

tests/test_extraction_agent.py 69-71행

```
def run_with(documents, proposals, plan=None):
    request = MagicMock(side_effect=[plan if plan is not None else plan_for(documents), {"items": proposals}])
    return agent.extract_upload(documents, request_fn=request), request
```

같은 파일 127-145행에서 2회 호출, 검색 결과의 다음 요청 전달, 원문·메타데이터 보존을 검사합니다. 가짜 응답 검사는 모델 정확도 평가가 아닙니다. 공지에 열거된 기술을 모두 사용했다고 표시하지 않고 적용 여부를 공개합니다.

5분 발표 구성

앞 2분은 주제·대표 결과, 뒤 3분은 실행 흐름·RAG 검색·근거 검증·비교·DI 코드를 설명합니다. 다음 두 쪽의 실제 코드를 본 발표의 코드 설명에 사용합니다.

06 실제 코드 설명 | 검색과 검증

검색 결과를 찾은 뒤 원문과 다시 대조했습니다

입력은 문서별 페이지 텍스트와 AI가 제안한 문서 ID·항목·검색어입니다. 아래는 현재 저장소에서 직접 읽은 실제 코드이며 전체 함수 중 핵심 부분입니다. 가독성을 위해 공통 들여쓰기만 제거했습니다.

키워드 점수와 원문 위치를 보존하는 검색

src/handoff_review/extraction_agent.py _search_documents 315-326행

```
        score = 1 + sum(term in text.casefold() for term in terms)
        ranked.append((score, occurrence))
    ranked.sort(key=lambda pair: (-pair[0], pair[1]["page"], pair[1]["start"]))
    for _, occurrence in ranked[:2]:
        text = occurrence["text"][:min(MAX_SNIPPET_CHARACTERS, query_budget)]
        if not text:
            continue
        budget -= len(text)
        query_budget -= len(text)
        hits.append({"document_id": query["document_id"], "field": query["field"],
                     "page": occurrence["page"], "text": text,
                     "chunk_id": f"chunk-{len(hits) + 1}"})
```

처리: 항목 별칭으로 찾은 원문 후보에 검색어 포함 점수를 부여하고, 점수·페이지·원문 위치 순으로 정렬했습니다. 검색 요청별 상위 2개를 문자 예산 안에서 반환합니다.
출력: 문서 ID, 항목, 페이지, 원문, 청크 ID입니다. 검색 실패는 정답이 없다는 뜻이 아니므로 미확정 항목을 임의 값으로 채우지 않습니다.

값과 단위가 인용문에도 존재하는지 검증

src/handoff_review/extraction_agent.py _validate_proposal 545-552행

```
        matching = [token for token in tokens if token == (proposal["value"], proposal["unit"])]
        quoted = [token for token in quote_tokens if token == (proposal["value"], proposal["unit"])]
    if not matching:
        return "AI 값·단위가 원문의 단일 결합 표기와 일치하지 않습니다."
    if not quoted:
        return "AI 인용에 제안된 값·단위가 함께 존재하지 않습니다."
    if not any(_direct_binding(field, value, unit, quote) for value, unit in quoted):
        return "AI 값·단위가 해당 항목명에 직접 연결된 단일 표기가 아닙니다."
```

입력: AI가 제안한 값·단위·인용문과 원문에서 읽은 토큰입니다. 숫자 항목은 추출 단계에서 단위를 바꿔 제안하지 못하도록 했습니다.
처리·보류: 원문 값·단위, 인용문 안의 값·단위, 항목명과의 직접 연결 중 하나라도 확인되지 않으면 거부 이유를 반환합니다. 통과해도 조건·복수값·출처 등 후속 검사가 남습니다.

07 실제 코드 설명 | 비교와 기술 선택

AI의 값 제안과 최종 판정을 분리했습니다

입력은 검증된 기준값·인계값과 각각의 출처입니다. 승인 변경 적용 순서와 고객사 확인은 이 비교 전에 검사하며, 여기서는 단위·조건을 맞춘 뒤 동일성을 비교합니다.

src/handoff_review/review.py 189-200행

```
        actual = normalize(spec, actual_entry)
        if actual_source is None:
            raise UncertainValue("인계값의 출처·인용문·양의 페이지 번호를 확인할 수 없습니다.")
        item["actual"] = {"value": actual.value, "unit": actual.unit, "source": actual_source}
        if expected.condition != actual.condition:
            raise UncertainValue("부품·운전조건 등 비교 조건이 같지 않거나 한쪽에서만 명시됐습니다.")
        if expected.comparable == actual.comparable:
            item.update(status="MATCH", reason="명시된 동일 항목·조건에서 단위 정규화 후 값이 같습니다.")
        else:
            item.update(status="MISMATCH", reason="명시된 동일 항목·조건에서 단위 정규화 후 값이 다릅니다.")
    except UncertainValue as exc:
        item.update(status="NEEDS_REVIEW", reason=str(exc))
```

출력: 조건이 같고 정규화값이 같으면 MATCH, 다르면 MISMATCH입니다. 출처나 조건이 불확실하면 NEEDS_REVIEW입니다. 확정 기준값은 있지만 인계값이 없는 MISSING 처리는 앞선 분기이며, 추출 실패와 실제 누락을 같은 의미로 취급하지 않습니다.

기술 선택 근거와 트레이드오프

선택	선택 이유	감수한 제한
OpenAI / Ollama	공통 검증·비교 코드를 두 실행 방식에서 재사용합니다.	OpenAI는 외부 처리·과금 동의가 필요합니다. 두 방식의 품질 우열은 측정하지 않았습니다.
키워드·별칭 검색	고정 10항목에서 검색 근거와 경로를 추적하기 쉽습니다.	임베딩 의미 검색이 아니며 새로운 표현을 놓칠 수 있습니다.
코드 기반 판정	단위·조건·변경 관계를 명시 규칙으로 비교했습니다.	지원하지 않는 단위·조건·모호한 값은 보류합니다.
로컬 웹·JSON 저장	설치 범위를 줄이고 검토 기록을 보존·복원했습니다.	다중 사용자 인증·운영 DB·공개 서비스 보안은 별도 과제입니다.

선택 과제 2의 도메인·문서 처리·근거 검증·판정·화면을 구현했습니다. 강의의 개념을 적용했지만 MCP·벡터 RAG·범용 PDF 이해를 구현했다고 주장하지 않습니다. 2회 호출은 처리 지연을 늘릴 수 있으며, 첫 검색 계획 호출의 추가 효용은 별도 비교로 입증하지 않았습니다.

부록 A 선택 비교 | 저장 응답 재생

판정은 유지하고 충돌 근거를 보완했습니다

기존 Ollama F01~F06 개발 기록 중 F04의 역사적 비교입니다. 가상 인계표에는 팬 전원이 400 V와 440 V로 함께 기재돼 있습니다. 이전 결과는 판정을 보류했지만 인계표 근거 하나만 연결했습니다. 보완 후에는 충돌하는 두 원문을 함께 보여주도록 했습니다.

동일 F04 입력의 비교 항목	Before	After
비교 방식	개발 중 보존한 이전 결과입니다.	2026-10-20 보완 코드로 저장된 응답을 재생했습니다.
팬 항목의 인계표 근거 수	1개	2개
팬 항목 evidence_complete	true	false
기술 판정 상태 보존	저장된 10개 상태	10/10개 동일
항목별 기준값·인계값 보존	저장된 값·단위·null	10/10개 동일

달라진 점

보류를 해제하지 않고 누락됐던 충돌 원문을 추가했습니다. 근거가 연결됐다는 사실과 판정을 확정할 수 있다는 사실을 분리했습니다. 판정은 보류로 유지하고 evidence_complete는 true에서 false로 바로잡았습니다.

측정 방법과 해석의 한계

당시 재생에서 기록된 모델 응답은 2회 사용했고 실제 모델 통신은 0회였습니다. 변경 전 코드를 다시 실행한 동시 실험이 아니라, 이전 저장 기준과 당시 보완 코드의 재생 결과를 대조한 개발 회귀 비교입니다. OpenAI 시험이 아닙니다.

같은 문서·같은 기록된 모델 응답·같은 항목 지표로 후처리의 차이를 확인했습니다. 보류가 늘거나 줄어드는 모델 성능 평가, 독립 정확도, 실제 AI 속도·업무 시간 절감의 개선으로 해석하지 않습니다. 사용한 F04는 독립 미사용 시험 자료가 아닙니다.

400 V와 440 V는 기존 모델 응답에 이미 있었습니다. 이번 차이는 AI 추출 능력이 아니라 근거 표시 보완입니다. 재생 요청은 당시 보완 코드가 만들었으며, 구버전 재실행이나 수업 보너스 조건 충족을 확정한 결과는 아닙니다.

측정 기록

reports/evidence-before-after-20261020T111053Z-4e8705ae.json

부록 B 보완 후 화면 | 전후 UI 쌍 비교 아님

보류된 두 원문을 함께 보여주는 화면입니다

기존 Ollama F04 보완 후에 저장한 실제 UI 캡처입니다. 고객 사양 400 V와 인계표의 400 V·440 V를 함께 표시하되 판정은 확인 필요로 유지했습니다. 수정 전 UI 캡처는 보존돼 있지 않습니다.

동일한 보완 후 캡처의 상태 요약과 두 인계표 근거를 확대 발췌했습니다.



같은 인계표 1쪽의 첫 번째 원문 후보



같은 인계표 1쪽의 두 번째 원문 후보



이 화면은 과거 보완 후 동작 증거이며 OpenAI 결과가 아닙니다. 앞쪽 표는 과거 저장 결과와 당시 코드의 응답 재생 비교이며, 구버전·신버전을 각각 새로 실행한 성능 비교나 전후 화면 비교가 아닙니다. 선택 보너스 충족을 확정하지 않습니다.